

Analysis of Adaptive Radix Tree (ART) Design and Range Query Implementation

Muyan Zhou

April 16, 2025

1 Design Logic of Adaptive Radix Tree (ART)

1.1 Core Design Principles and Motivation

The Adaptive Radix Tree (ART) is a high-performance in-memory indexing structure that addresses the limitations of traditional radix trees and Tries. Its core design principles include:

- **Adaptive Node Sizes:** ART uses four different node types (`Node4`, `Node16`, `Node48`, `Node256`) that automatically adapt to the number of children, as shown in the `ArtNodeType` enum in `art.cpp`. By using different node types, ART minimizes memory overhead while maintaining fast access times.
- **Path Compression:** ART employs prefix compression to reduce tree height, implemented via the `prefix` and `prefixLength` fields in the `ArtNode` structure. Also, nodes with only one child are compressed, reducing tree height and improving lookup times.

1.2 Improvements Over Traditional Structures

ART improves upon traditional radix trees and Tries in several ways:

- **Memory Efficiency:** Unlike fixed-size nodes in conventional radix trees, ART's adaptive nodes reduce memory consumption for sparse nodes. ART automatically upgrades/downgrades node types during insertion/deletion (implemented in `insertNode*` and `eraseNode*` functions).
- **Time Efficiency:** ART's adaptive node types and path compression lead to faster lookups and insertions. For example, the `lookup` function uses SIMD operations for `Node16` and direct indexing for `Node256`, significantly speeding up key comparisons.

1.3 Efficient Operations

ART supports efficient operations through:

- **Insertion:** The `insert` function handles prefix splitting and node type conversion. It uses path compression and maintains sorted keys for efficient child lookup.
- **Lookup:** The `lookup` function leverages prefix compression and optimized child search methods (e.g. SIMD in Node16, direct indexing in Node256).
- **Deletion:** The `erase` function handles node consolidation and type downgrading when child counts fall below thresholds.

2 Implementation of Range Query

At the beginning, the range is made inclusive to simplify the implementation. The range query implementation in `rangeScan` employs an optimized depth-first traversal with the following key aspects:

1. **Prefix Accumulation:** Maintains a running prefix during traversal that represents the path from the root to the current node.
2. **Three-State Range Checking:**
 - `equalLow`: True if current path exactly matches lower bound prefix
 - `equalHigh`: True if current path exactly matches upper bound prefix
 - `in_range`: Integer flag indicating position relative to range (1=inside, 0=border, -1=outside)
3. **Early Termination:**
 - Skips subtrees when `in_range` is -1
 - Fast-tracks collection when `in_range` is 1
4. **Leaf Validation:**
 - For leaves, performs exact key comparison against bounds
 - Only adds to results if $\text{key} \in [\text{lowerKey}, \text{upperKey}]$